Algorithms, programming and logic

See section 4 for the:

- standard flowchart symbols that must be used by students when drawing flowcharts
- logic gate symbols that must be used by students when drawing logic circuits
- format in which pseudocode will appear in examinations.

In Paper 2, candidates are expected to write solutions to given problems using the methods and concepts listed in topics 7 to 10.

Where the solution to a problem involves coding, candidates are required to write solutions in pseudocode. Solutions written in programming code will **not** be awarded marks. The exception is the 15-mark scenario question.

In the 15-mark scenario question, candidates are required to write solutions to the given problem using pseudocode or their choice of one of the following programming languages: Python, Visual Basic or Java. Solutions written in other programming languages will **not** be awarded marks.

7 Algorithm design and problem-solving

Candidates should be able to:	Notes and guidance
1 Understand the program development life cycle, limited to: analysis, design, coding and testing	• Including identifying each stage and performing these tasks for each stage: – analysis: abstraction, decomposition of the problem, identification of the problem and requirements – design: decomposition, structure diagrams, flowcharts, pseudocode – coding: writing program code and iterative testing – testing: testing program code with the use of test data
2 (a) Understand that every computer system is made up of sub-systems, which are made up of further sub-systems	
(b) Understand how a problem can be decomposed into its component parts	• Including: – inputs – processes – outputs – storage
(c) Use different methods to design and construct a solution to a problem	• Including: – structure diagrams – flowcharts – pseudocode
3 Explain the purpose of a given algorithm	• Including: – stating the purpose of an algorithm – describing the processes involved in an algorithm

7 Algorithm design and problem-solving continued

Candidates should be able to:	Notes and guidance
4 Understand standard methods of solution	• Limited to: – linear search – bubble sort – totalling – counting – finding maximum, minimum and average values
5 (a) Understand the need for validation checks to be made on input data and the different types of validation check	• Including: – range check – length check – type check – presence check – format check – check digit
(b) Understand the need for verification checks to be made on input data and the different types of verification check	• Including: – visual check – double entry check
6 Suggest and apply suitable test data	• Limited to: – normal – abnormal – extreme – boundary • Extreme data is the largest/smallest acceptable value • Boundary data is the largest/smallest acceptable value and the corresponding smallest/largest rejected value
7 Complete a trace table to document a dry-run of an algorithm	• Including, at each step in an algorithm: – variables – outputs – user prompts
8 Identify errors in given algorithms and suggest ways of correcting these errors	
9 Write and amend algorithms for given problems or scenarios, using: pseudocode, program code and flowcharts	• Precision is required when writing algorithms, e.g. `x > y` is acceptable but `x is greater than y` is not acceptable • See section 4 for flowchart symbols • See section 4 for pseudocode